

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

КУРСОВА РОБОТА
(ПОЯСНЮВАЛЬНА ЗАПИСКА)

з дисципліни: «Технології машинного навчання та штучний інтелект»
на тему: **«Розробка та порівняльний аналіз методів пояснюваного ШІ-
оцінювання студентських програмних робіт»**

студента 1 курсу групи _ІПЗм-25-1
Галузь знань 12 «Інформаційні технології»
спеціальність F2 «Інженерія програмного
забезпечення»

ступінь «**магістр**»

Осипчука Антона Олексійовича

(прізвище, ім'я та по-батькові)

Керівник ст. викл. каф. КН Марчук Г.В.

Дата захисту: «__» _____ 20__ р.

Національна оцінка: _____

Кількість балів: _____

Оцінка: ECTS _____

Члени комісії:

_____ Андрій МОРОЗОВ
(підпис) (прізвище та ініціали)

_____ Галина МАРЧУК
(підпис) (прізвище та ініціали)

_____ Дмитро ПЛЕЧІСТИЙ
(підпис) (прізвище та ініціали)

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення
Освітній рівень: магістр
Спеціальність F2 «Інженерія програмного забезпечення»

«ЗАТВЕРДЖУЮ»
Завідувач кафедри

«__» _____ 202__ р.

ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ
Осипчуку Антону Олексійовичу

1. Тема роботи: «Розробка та порівняльний аналіз методів пояснюваного ШІ-оцінювання студентських програмних робіт»,
керівник роботи: Галина МАРЧУК
2. Строк подання студентом: “01” червня 2026 р.
3. Вихідні дані до роботи:

Об'єктом дослідження є процес автоматизованого та ШІ оцінювання студентських програмних робіт.

Предметом дослідження є методи та інструментальні засоби пояснюваного оцінювання програмного коду, зокрема rule-based оцінювання на основі тест-кейсів, зважені тест-кейси, статичний аналіз коду, LLM-based оцінювання, test-aware LLM-підхід та метод детерміністичних обмежень.

4. Перелік графічного матеріалу(з точним зазначенням обов'язкових креслень)
 1. Презентація до курсової роботи
 2. Посилання на репозиторій:
<https://github.com/AntonOsypchuk1/AIGradingService>
5. Дата видачі завдання “ 17 ” лютого 2026 р

Магістрант **Осипчук Антон**

Керівник **Галина Марчук**

РЕФЕРАТ

Пояснювальна записка до курсової роботи «Розробка та порівняльний аналіз методів пояснюваного AI-assisted оцінювання студентських програмних робіт» складається зі вступу, трьох розділів, висновків, списку використаних джерел та додатків.

Текстова частина викладена на 26 сторінках друкованого тексту. Список використаних джерел містить 12 найменувань і займає 1 сторінку. Пояснювальна записка має 9 сторінок додатків. У роботі наведено 1 рисунок та 3 таблиць. Загальний обсяг роботи – 43 сторінки.

У першому розділі розглянуто постановку задачі пояснюваного ІІІ оцінювання студентських програмних робіт. Проаналізовано основні підходи до автоматизованого оцінювання програмного коду, та обґрунтовано вибір технологій для реалізації сервісу оцінювання.

У другому розділі описано формування контрольного набору даних на основі навчальних лабораторних завдань з Python. Спроектовано архітектуру сервісу ІІІ оцінювання, реалізовано порядок виконання студентського коду, запуску тестів, розрахунку метрик та порівняння різних підходів.

У третьому розділі проведено експериментальний аналіз якості реалізованих методів оцінювання. Було порівняно equal-weight rule-based, weighted rule-based, static-analysis, hybrid rule-based, LLM-only, test-aware LLM та guarded LLM підходи.

У додатках наведено фрагменти програмного коду, структуру контрольного набору даних, приклади API-відповідей, результати експериментів та додаткові матеріали до реалізованого сервісу.

Ключові слова: AI-ASSISTED ОЦІНЮВАННЯ, ПОЯСНЮВАНИЙ ІІІ, PYTHON, .NET, ASP.NET CORE, LLM, OPENROUTER STATIC ANALYSIS.

					ІІЗм.КР.Б-125-26-ПЗ						
Змн.	Арк.	№ докум.	Підпис	Дата							
Розроб.		Осипчук А.О.			Розробка та порівняльний аналіз методів пояснюваного AI-assisted оцінювання студентських програмних робіт			Лім.		Арк.	Аркушів
Перевір.		Марчук Г.В.								3	43
Керівник								Житомирська політехніка, група ІІЗм-25-1			
Н. контр.											
Зав. каф.											

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ

ML (Machine Learning) – машинне навчання;

LLM (Large Language Model) – велика мовна модель;

API (Application Programming Interface) – інтерфейс програмування додатків;

JSONL (JSON Lines) – формат збереження даних, у якому кожен рядок є окремим JSON-об'єктом;

CSV (Comma-Separated Values) – текстовий формат табличних даних із розділенням значень комами;

Baseline – базовий метод або модель, з якою порівнюються інші підходи;

Guardrails – детерміновані обмеження, які зменшують ризик некоректної або небезпечної відповіді AI-системи;

MAE (Mean Absolute Error) – середня абсолютна помилка між очікуваною та прогнозованою оцінкою;

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

ЗМІСТ

ВСТУП	6
РОЗДІЛ 1. АНАЛІЗ ЗАДАЧІ, МЕТОДІВ ТА ЗАСОБІВ МАШИННОГО НАВЧАННЯ.....	8
1.1 Постановка задачі оцінювання студентських програмних робіт.....	8
1.2 Огляд методів автоматизованого оцінювання програмного коду	9
1.3 Огляд інструментів та технологій реалізації.....	13
Висновки до першого розділу.....	14
РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ СЕРВІСУ ШІ ОЦІНЮВАННЯ.....	15
2.1 Характеристика та підготовка даних	15
2.2 Архітектура сервісу оцінювання	17
2.3 Реалізація моделей оцінювання.....	20
Висновки до другого розділу	24
РОЗДІЛ 3. АНАЛІЗ ЯКОСТІ МЕТОДІВ ОЦІНЮВАННЯ	25
3.1 Метрики якості та їх обґрунтування	25
3.2 Результати та порівняльний аналіз моделей	26
3.3 Аналіз помилок та інтерпретація	28
3.4 Обмеження роботи та напрями розвитку	29
Висновки до третього розділу	30
ВИСНОВКИ.....	31
СПИСОК ЛІТЕРАТУРИ.....	33
ДОДАТКИ.....	34

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

ВСТУП

У сучасному освітньому процесі зростає потреба в інструментах, які можуть частково автоматизувати перевірку студентських програмних робіт та зменшити навантаження на викладача. Особливо актуальною ця задача є для дисциплін з програмування, де кожна лабораторна робота може містити значну кількість подань, що потребують перевірки коректності, повноти реалізації, якості коду та відповідності умові завдання. Традиційна ручна перевірка є трудомісткою, а звичайні автоматичні тести не завжди дозволяють повністю оцінити якість розв'язку, оскільки вони перевіряють лише поведінку програми на визначених вхідних даних.

Одним із перспективних напрямів вирішення цієї проблеми є використання методів штучного інтелекту та великих мовних моделей для підтримки процесу оцінювання. Такі моделі здатні аналізувати програмний код, пояснювати помилки, порівнювати реалізацію з умовою задачі та формувати попередню оцінку за заданою рубрикою. Водночас використання LLM у задачах оцінювання має певні ризики: модель може переоцінювати неправильні рішення, ігнорувати результати тестів або повертати нестабільні відповіді. Тому доцільним є поєднання декількох підходів: автоматичних тестів, зважених тест-кейсів, статичного аналізу, LLM-оцінювання та детерміновані обмежувачів.

Мета роботи – розробка та порівняльний аналіз методів пояснюваного III оцінювання студентських програмних робіт з використанням різноманітних підходів, статичного аналізу, результатів автоматичних тестів та великих мовних моделей.

Об'єкт дослідження – процес автоматизованого та III оцінювання студентських програмних робіт.

Предмет дослідження – методи та інструментальні засоби пояснюваного оцінювання програмного коду на основі тест-кейсів, статичного аналізу, LLM підходів і обмежувачів.

Для досягнення мети поставлено такі завдання:

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

1. Проаналізувати задачу автоматизованого оцінювання студентських програмних робіт та визначити її основні обмеження.
2. Розглянути підходи до оцінювання програмного коду: автоматичні тести, зважені тест-кейси, статичний аналіз, гібридні rule-based методи та LLM оцінювання.
3. Сформувати контрольний набір даних на основі навчальних лабораторних завдань з Python.
4. Спроектувати архітектуру сервісу III оцінювання з підтримкою різних моделей.
5. Провести експериментальний аналіз реалізованих методів.

Практична цінність роботи полягає у створенні прототипу сервісу пояснюваного III оцінювання студентських програмних робіт, який може використовуватися як основа для подальшої інтеграції в освітні системи. Розроблений сервіс дозволяє запускати програмний код на тест-кейсах, порівнювати різні методи оцінювання, формувати пояснення до оцінки та позначати ризикові випадки, які потребують обов'язкової перевірки викладачем.

		Осипчук А.О.			ІІІЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 1. АНАЛІЗ ЗАДАЧІ, МЕТОДІВ ТА ЗАСОБІВ МАШИННОГО НАВЧАННЯ

1.1 Постановка задачі оцінювання студентських програмних робіт

Автоматизоване оцінювання студентських програмних робіт є актуальною задачею для освітніх інформаційних систем, оскільки перевірка коду вручну потребує значних часових витрат викладача, особливо за наявності великої кількості студентських подань. У межах навчальних дисциплін з програмування типова лабораторна робота містить набір практичних завдань, для яких студент має розробити програмне рішення, що відповідає умові задачі, коректно обробляє вхідні дані та формує очікуваний результат.

Традиційним підходом до автоматизації перевірки програмного коду є використання тест-кейсів. Такий підхід дозволяє швидко перевірити функціональну коректність програми: система запускає код студента на наборі вхідних даних і порівнює фактичний результат із очікуваним. Однак лише автоматичних тестів часто недостатньо для повноцінного оцінювання. Програма може пройти частину тестів випадково, але мати неправильну логіку; інша програма може бути частково правильною, але не відповідати вимогам до стилю, структури або повноти реалізації. Крім того, звичайні тести не пояснюють, чому саме виставлена та чи інша оцінка.

У цій роботі розглядається задача розробки сервісу, який формує попередню оцінку студентської програмної роботи на основі кількох джерел інформації: умови завдання, коду студента, результатів автоматичних тестів, статичного аналізу та оцінювання за допомогою великої мовної моделі. Важливо, що система не повинна замінювати викладача повністю. Її призначення – надати попередню оцінку, пояснення, виявлені помилки та прапорці ризику для подальшого перегляду людиною.

З формальної точки зору задача може бути подана як задача оцінювання з використанням штучного інтелекту, тобто автоматизованого прогнозування оцінки за програмне подання. На відміну від класичної класифікації, де результат

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

належить до одного з наперед заданих класів, у цій роботі прогнозується числова оцінка в межах шкали від 0 до 10 балів. Тому задачу можна інтерпретувати як наближену регресійну задачу з додатковим пояснюваним виходом.

Вхідними даними системи є:

- умова програмного завдання;
- код студентського розв'язку мовою Python;
- набір тест-кейсів;
- очікувана оцінка для контрольного порівняння;
- рубрика оцінювання.

Вихідними даними системи є:

- прогнозована оцінка;
- оцінки за окремими критеріями: коректність, повнота, якість коду;
- пояснення до кожного критерію;
- результати виконання тестів;
- прапорці ризику;
- політика фінального рішення.

Критеріями успішності вирішення задачі є не тільки близькість прогнозованої оцінки до очікуваної, а й здатність системи пояснити результат та виявити ризикові випадки, які обов'язково мають бути переглянуті викладачем.

1.2 Огляд методів автоматизованого оцінювання програмного коду

Для розв'язання поставленої задачі можна використати кілька груп методів. Кожен із них має власні переваги та обмеження, тому в межах роботи доцільно не обмежуватися одним підходом, а провести їх порівняльний аналіз. Це відповідає вимогам методичних рекомендацій, де зазначено, що в першому розділі необхідно розглянути декілька методів, проаналізувати їх переваги, недоліки та обґрунтувати вибір підходу для практичної реалізації.

Найпростішим і найбільш поширеним способом автоматизованої перевірки програмних робіт є запуск коду на наборі тест-кейсів. Для кожного

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

тесту задаються вхідні дані та очікуваний результат. Якщо фактичний результат програми збігається з очікуванням, тест вважається пройденим.

У загальному вигляді оцінку можна обчислити за формулою:

$$Score = \frac{PassedTests}{TotalTests} * MaxScore,$$

де PassedTests – кількість пройдених тестів, TotalTests – загальна кількість тестів, MaxScore – максимальна оцінка за завдання.

Перевагами такого підходу є простота, швидкість і відтворюваність. Один і той самий код за однакових тестів завжди дає однаковий результат. Це робить тестовий підхід зручним для базової автоматичної перевірки. Недоліком є залежність якості оцінювання від якості тест-кейсів. Якщо тестів мало або вони не охоплюють важливі граничні випадки, система може переоцінити неправильне рішення.

Покращенням звичайного тестового підходу є використання зважених тест-кейсів. У цьому випадку кожному тесту призначається вага, що відображає його важливість для перевірки ключової логіки задачі. Наприклад, для задачі про високосний рік тест 1900 = normal має більшу вагу, ніж простіший тест 2024 = leap, оскільки перевіряє виняток із правила.

Формула зваженого проходження тестів має вигляд:

$$WeightedPassRate = \frac{\sum PassedTestsWeights}{\sum TotalTestsWeights}$$

Тоді оцінка може бути розрахована так:

$$Score = WeightedPassRate * MaxScore,$$

Перевагою такого підходу є те, що він краще враховує важливість граничних випадків. Якщо студентське рішення не проходить ключовий тест, воно отримує нижчу оцінку, навіть якщо проходить простіші перевірки. Недоліком є необхідність ручного визначення ваг. Якщо ваги підібрані некоректно, система може або занадто сильно штрафувати частково правильні рішення, або навпаки недостатньо знижувати оцінку за критичні помилки.

Окрім вище наведених методів перевірки коду за тест-кейсами, також потрібно проводити статичний аналіз коду. На цьому етапі ми не виконуємо

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		10

програму, а перевіряємо її структуру. У межах цієї роботи такий аналіз використовується для виявлення наявності необхідних програмних конструкцій: input, if, elif, циклів, оператора %, роботи зі списками, функцій max, split, count тощо.

Недолік полягає в тому, що наявність певних конструкцій у коді не гарантує правильну поведінку програми. Студент може використати if, for або max, але все одно реалізувати неправильну логіку. Тому статичний аналіз не є достатнім як самостійний метод оцінювання, але може бути корисним як допоміжний сигнал.

На даному етапі ми підходимо до гібридного підходу на основі правил, що поєднує результати зважених тестів і статичного аналізу. Основою оцінювання є поведінкова перевірка через тести, а статичний аналіз використовується для додаткових штрафів або прапорців ризику. Такий підхід є більш стійким, ніж окремі методи, оскільки враховує і фактичну поведінку програми, і її структуру.

Попри стабільність детермінованих алгоритмів і гібридних правил, вони не здатні оцінити логіку коду на семантичному рівні та надати розгорнуте текстове пояснення оцінки. Для подолання цих обмежень та забезпечення інтерпретованості результатів доцільно залучити інструменти генеративного штучного інтелекту. Першим серед таких методів є LLM-only підхід, за якого велика мовна модель аналізує лише умову задачі та код студента, формуючи оцінку за заданою рубрикою.

LLM-only підхід передбачає, що велика мовна модель отримує тільки умову задачі та код студента, після чого формує оцінку за заданою рубрикою. Перевагою такого підходу є здатність моделі аналізувати логіку коду, формулювати пояснення та виявляти помилки, які не завжди очевидні з тестових результатів.

Проте цей підхід має суттєві ризики. Модель може переоцінити неправильне рішення, не помітити граничний випадок або згенерувати оцінку, яка не відповідає фактичній поведінці програми. Крім того, LLM може повертати

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

відповідь у неправильному форматі, що створює додаткові технічні проблеми під час автоматичної обробки.

Більш надійним підходом є test-aware LLM оцінювання. У цьому випадку модель отримує не лише умову задачі та код студента, а й результати автоматичних тестів: кількість пройдених тестів, зважену частку проходження, провалені кейси, очікувані та фактичні результати, повідомлення про помилки.

Це дозволяє моделі формувати пояснення на основі конкретних доказів. Наприклад, якщо програма не проходить тест для значення 1900, модель може пояснити, що студент не врахував правило для столітніх років. Таким чином, LLM не просто «оцінює код», а інтерпретує результати виконання програми.

Недоліком є те, що навіть за наявності тестових результатів модель може не завжди правильно їх враховувати. Деякі моделі можуть переоцінювати рішення, яке має помилку виконання або не проходить важливий тест. Тому для практичного використання доцільно додавати детерміновані обмежувачі.

Детерміновані обмежувачі – це набір таких обмежень, що застосовуються після оцінювання LLM. Наприклад, якщо програма не пройшла жодного тесту, фінальна оцінка не повинна перевищувати 2 бали. Якщо є помилка виконання, оцінка не повинна перевищувати 4 бали. Якщо weighted pass rate нижчий за 0.5, оцінка не повинна бути надто високою.

Такий підхід не замінює LLM, а обмежує її ризикову поведінку. Він особливо важливий для освітніх систем, де автоматична оцінка може впливати на навчальні результати студента. У межах цієї роботи детерміністичні обмеження використовуються як механізм зменшення небезпечного переоцінювання.

Порівняння розглянутих підходів наведено в таблиці 1.1.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		12

Методи автоматизованого оцінювання програмних робіт

Метод	Основна ідея	Переваги	Недоліки
Equal-weight tests	Усі тести мають однакову вагу	Простота, швидкість, відтворюваність	Не враховує важливість edge cases
Weighted tests	Тести мають різну вагу	Краще оцінює критичні випадки	Потребує ручного налаштування ваг
Static analysis	Аналіз структури коду без виконання	Виявляє відсутні конструкції та ризики	Не гарантує правильну поведінку
Hybrid rule-based	Поєднання тестів і статичного аналізу	Більш стійкий deterministic baseline	Залежить від правил і евристик
LLM-only	Оцінювання коду за допомогою LLM	Дає пояснення, аналізує логіку	Може галюцинувати або переоцінювати
Test-aware LLM	LLM враховує результати тестів	Кращі пояснення на основі доказів	Залежить від якості моделі та prompt
Guarded LLM	LLM + deterministic caps	Зменшує ризик небезпечного переоцінювання	Може бути занадто консервативним

Для практичної реалізації в цій роботі обрано порівняльний підхід: реалізуються кілька підходів, після чого їх якість оцінюється за єдиними метриками. Такий формат дозволяє не просто показати роботу одного алгоритму, а критично порівняти методи та визначити, у яких випадках кожен із них є доцільним.

1.3 Огляд інструментів та технологій реалізації

Для реалізації сервісу оцінювання студентських програмних робіт було обрано технологічний стек, орієнтований на інтеграцію з уже наявною архітектурою програмних систем та можливість подальшого використання як окремого мікросервісу.

Основною платформою реалізації обрано .NET / ASP.NET Core Web API. Такий вибір обумовлений тим, що .NET добре підходить для побудови backend-сервісів, має розвинену екосистему для створення API та може бути інтегрований з існуючими освітніми або адміністративними системами. У межах роботи Web API використовується для запуску оцінювання, перегляду набору завдань, виконання тестів і порівняння підходів.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

Для виконання студентського Python-коду використовується окремий компонент PythonExecutionService. Його задача – створити тимчасовий файл із кодом студента, передати вхідні дані, виконати програму через Python runtime, отримати stdout/stderr і порівняти результат із очікуваним. Такий підхід дозволяє перевіряти поведінку коду на контрольному наборі тест-кейсів.

Основний процес оцінювання виконує EvaluationPipelineService. Його задача – взяти подання, знайти відповідне завдання, запустити тести, передати результат у потрібний підхід і повернути структурований результат оцінювання. Для порівняння методів використовується EvaluationMetricsService, який обчислює агреговані метрики якості.

Для роботи з великими мовними моделями використовується абстракція ILlmClient. Це дозволяє підключати різні джерела LLM без зміни підходів. У роботі використано OpenRouter API як спосіб доступу до хмарних LLM-моделей. Такий підхід є зручним для експериментів, оскільки дозволяє порівнювати різні моделі через спільний API.

Висновки до першого розділу

У цьому розділі було формалізовано задачу пояснюваного III оцінювання студентських програмних робіт, визначено вхідні та вихідні дані, описано основні групи методів автоматизованої перевірки коду та обґрунтовано вибір порівняльного підходу. Подальший розділ присвячено проектуванню та реалізації сервісу, який дозволяє практично порівняти розглянуті методи на контрольному наборі програмних подань.

		Осипчук А.О.			ІІЗМ.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		14

РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ СЕРВІСУ ШІ ОЦІНЮВАННЯ

2.1 Характеристика та підготовка даних

Для практичної перевірки методів оцінювання у роботі було сформовано контрольний набір даних на основі навчальних лабораторних робіт з мови програмування Python. Використання такого набору даних дозволяє перевірити роботу сервісу в умовах, близьких до реального освітнього процесу, але без використання персональних або конфіденційних студентських даних. Завдання було взято з навчальних лабораторних робіт, а варіанти студентських подань створено вручну для моделювання різних рівнів якості розв'язку.

Основою для набору даних стали завдання з лабораторних робіт з Python. Зокрема, у лабораторних роботах розглядаються розгалуження та цикли, включно із задачею визначення високосного року з урахуванням правил подільності на 4, 100 і 400; робота з рядками, зокрема пошук, розбиття тексту та підрахунок слів; списки, зокрема задача пошуку максимального елемента та виведення списку у зворотному порядку.

Для першої версії датасету було відібрано чотири завдання:

1. L2_T2 – визначення високосного року;
2. L2_T3 – розрахунок вартості покупки з урахуванням знижки;
3. L3_T8 – підрахунок кількості входжень заданого слова у тексті;
4. L4_T1 – пошук максимального елемента списку та виведення списку у зворотному порядку.

Кожне завдання представлено у файлі assignments.csv. Для кожного завдання зберігаються ідентифікатор, номер лабораторної роботи, тема, текст умови, максимальна оцінка та перелік очікуваних програмних концептів, які можуть бути використані під час статичного аналізу.

Для кожного завдання було створено декілька варіантів студентських подань:

- correct – повністю правильне рішення;
- partial – частково правильне рішення;

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

- wrong – рішення з неправильною логікою;
- runtime/syntax error – рішення, яке не виконується або містить критичну помилку.

Ці подання збережено у файлі submissions.jsonl. Формат JSONL обрано тому, що кожен рядок містить окремий JSON-об'єкт, а код студента може містити переноси рядків, лапки та спеціальні символи. Такий формат зручніший для збереження програмного коду, ніж звичайний CSV.

Приклад запису у submissions.jsonl:

```
{
  "submission_id": "S_L2_T2_001",
  "assignment_id": "L2_T2",
  "variant_type": "correct",
  "student_code": "year = int(input('Enter a year: '))\nif year % 400 == 0 or (year % 4 == 0 and\nyear % 100 != 0):\n print('leap')\nelse:\n print('normal')",
  "expected_score": 10
}
```

Поле expected_score містить умовну експертну оцінку, яка використовується для обчислення метрик якості. Важливо зазначити, що цей dataset не подається як реальний набір студентських робіт. Він є контрольним набором програмних подань, сформованим для перевірки роботи різних методів оцінювання.

Окремо було створено файл test_cases.csv, у якому для кожного завдання визначено набір тестів. Кожен тест містить вхідні дані, очікуваний результат і вагу. Вага дозволяє відобразити важливість конкретного тесту. Наприклад, для задачі визначення високосного року тест із роком 1900 має більшу вагу, оскільки перевіряє виняток для років, кратних 100, але не кратних 400.

Приклад структури test_cases.csv:

```
assignment_id,input_data,expected_output,weight
L2_T2,2000,leap,2
L2_T2,1900,normal,4
L2_T2,2010,normal,2
L2_T2,2024,leap,2
```

У роботі також використовується файл rubric.json, який описує критерії оцінювання:

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		16

correctness – функціональна правильність розв’язку;

completeness – повнота реалізації вимог задачі;

code_quality – якість, читабельність та структурованість коду.

Загальна оцінка формується за шкалою від 0 до 10 балів. У базовій рубриці критерій correctness має максимальну вагу 5 балів, completeness – 3 бали, code_quality – 2 бали.

Таким чином, контрольний набір даних містить чотири задачі, шістнадцять варіантів подань і набір тест-кейсів із вагами. Цього достатньо для первинної перевірки архітектури сервісу, порівняння моделей і виявлення типових проблем автоматизованого оцінювання.

2.2 Архітектура сервісу оцінювання

Практична реалізація виконана у вигляді окремого backend-сервісу на платформі .NET / ASP.NET Core Web API. Такий підхід дозволяє розглядати розроблений компонент як незалежний сервіс, який у майбутньому може бути інтегрований у більшу систему оцінювання студентських робіт або використаний як окремий мікросервіс.

Загальна схема роботи сервісу представлена на рис. 1.

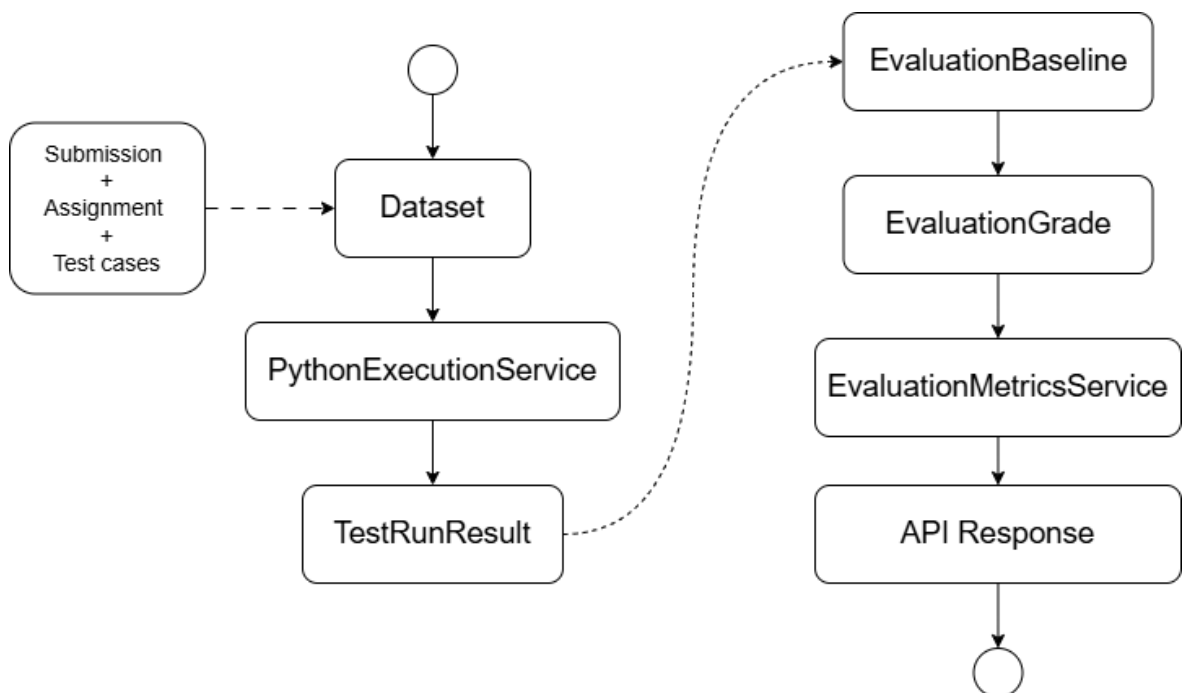


Рис. 1. Схема роботи сервісу

Основними компонентами сервісу є:

1. DatasetService – завантажує завдання, подання та тест-кейси з файлів.
2. PythonExecutionService – виконує Python-код студента на тест-кейсах.
3. IEvaluationBaseline – спільний інтерфейс для всіх методів оцінювання.
4. EvaluationPipelineService – організовує запуск оцінювання для одного submission або всього dataset.
5. EvaluationMetricsService – обчислює агреговані метрики.
6. ILlmClient – абстракція для роботи з LLM.
7. OpenRouterLlmClient – реалізація LLM-клієнта через OpenRouter API.
8. GradingGuardrailService – застосовує deterministic guardrails до оцінок LLM.

Компонент DatasetService відповідає за завантаження контрольного набору даних. При запуску сервіс читає файли assignments.csv, submissions.jsonl і test_cases.csv, після чого зберігає їх у пам'яті застосунку. Для MVP-версії було обрано in-memory підхід, оскільки він спрощує розробку та дозволяє швидко перевірити логіку pipeline без налаштування бази даних.

Компонент PythonExecutionService виконує студентський код на тестових даних. Для кожного submission сервіс створює тимчасовий .py файл, передає вхідні дані через стандартний ввід, отримує результат зі стандартного виводу та порівнює його з очікуваним результатом.

Результат запуску зберігається в об'єкті TestRunResult (див. Додаток А), який містить кількість пройдених тестів, загальну кількість тестів, суму ваг пройдених тестів, загальну суму ваг, список результатів окремих тест-кейсів, ознаку помилки виконання або синтаксичної помилки.

Для порівняння результатів використано не повний збіг рядків, а перевірку на наявність очікуваних фрагментів. Це потрібно тому, що студентські програми часто виводять додаткові підказки, наприклад Enter a year:, Max value: або Reverse:. Для складених очікуваних результатів використовується роздільник «|», який означає, що фактичний вивід має містити всі зазначені частини.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				18
Змн.	Арк.	№ докум.	Підпис	Дата		

Для того щоб реалізувати кілька методів оцінювання без дублювання логіки, було створено спільний інтерфейс IEvaluationBaseline:

```
public interface IEvaluationBaseline
{
    string Name { get; }

    Task<EvaluationGrade> EvaluateAsync(
        BaselineEvaluationContext context,
        CancellationToken cancellationToken = default);
}
```

Кожен baseline отримує однаковий контекст оцінювання:

```
public class BaselineEvaluationContext
{
    public Assignment Assignment { get; set; } = null!;
    public Submission Submission { get; set; } = null!;
    public TestRunResult TestRun { get; set; } = null!;
}
```

Це дозволяє однаково запускати як rule-based методи, так і LLM-based методи. Усі baseline-и повертають спільний результат EvaluationGrade (див. Додаток А). Наявність поля FinalDecisionPolicy є важливою, оскільки розроблений сервіс не приймає фінальне рішення замість викладача. Він формує попередню оцінку, пояснення та ознаки ризику, а фінальне рішення має бути підтверджене людиною.

Компонент EvaluationPipelineService відповідає за запуск оцінювання. Він будує контекст для конкретного подання: знаходить завдання, завантажує тест-кейси, запускає Python-код і передає результат обраному підходу.

Сервіс підтримує два режими роботи:

- оцінювання одного подання одним підходом;
- оцінювання всього датасету одним або кількома підходами.

Це дозволяє запускати як точкове тестування, так і повний експеримент.

Окремо реалізовано обробку помилок на рівні окремого подання. Якщо один LLM-запит не виконався або модель повернула некоректний формат, це не повинно зупиняти весь експеримент. У такому випадку система створює результат із прапорцем evaluation_failed, а решта подань продовжують оцінюватися.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		19

Для роботи з великими мовними моделями було створено інтерфейс ILlmClient:

```
public interface ILlmClient
{
    string ModelName { get; }

    Task<string> CompleteAsync(
        string prompt,
        CancellationToken cancellationToken = default);
}
```

Така абстракція дозволяє замінити конкретного постачальника LLM без зміни підходів. У роботі реалізовано FakeLlmClient для перевірки архітектури та OpenRouterLlmClient для реального підключення до мовних моделей через OpenRouter API.

Для стабільної роботи з API додано retry policy:

- повтор запиту при помилці 429 Too Many Requests;
- до трьох спроб;
- очікування згідно з Retry-After header або стандартна пауза;
- таймаут HTTP-клієнта для довгих відповідей.

Також реалізовано повторну спробу при помилках форматування відповіді LLM. Якщо модель повертає невалідний JSON або markdown замість чистого JSON, сервіс формує уточнений промпт і повторює запит.

2.3 Реалізація моделей оцінювання

У роботі реалізовано сім різних моделей оцінювання. Їхня мета – порівняти різні підходи до виставлення попередньої оцінки за програмне подання.

Першим baseline є equal_weight_rule_based. Він використовує лише кількість пройдених тестів без урахування їхньої важливості.

Оцінка формується за трьома критеріями:

- correctness – до 5 балів;

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				20
Змн.	Арк.	№ докум.	Підпис	Дата		

- completeness – до 3 балів;
- code_quality – до 2 балів.

Для correctness використовується частка пройдених тестів:

$$PassRate = \frac{PassedTests}{TotalTests}$$

$$CorrectnessScore = PassRate * 5$$

Completeness оцінюється евристично на основі pass rate. Якщо пройдено всі тести, submission отримує максимальну повноту; якщо пройдено частину тестів – часткову оцінку. Code quality визначається за простими евристиками: наявність input, довжина коду, відсутність потенційно проблемних конструкцій.

Перевага цієї моделі – простота та повна відтворюваність. Недолік – усі тести мають однакову вагу, тому складні edge cases не мають більшого впливу на оцінку.

Друга модель – weighted_rule_based. Вона працює аналогічно попередній, але враховує ваги тест-кейсів. Зважена частка проходження тестів обчислюється так:

$$WeightedPassRate = \frac{PassedWeight}{TotalWeight}$$

$$CorrectnessScore = WeightedPassRate * 5$$

Цей підхід дозволяє сильніше штрафувати рішення, які не проходять ключові перевірки. Наприклад, у задачі визначення високосного року тест із роком 1900 має більшу вагу, оскільки перевіряє суттєву частину алгоритму.

Weighted є більш точним, ніж equal-weight, якщо ваги тестів підібрано відповідно до логіки задачі. Водночас він залежить від ручного налаштування ваг.

Третя модель – static_analysis_rule_based. Вона не використовує результати тестів як основне джерело оцінки, а аналізує структуру коду. Для кожного завдання у assignments.csv визначено список необхідних концептів.

Сервіс перевіряє, чи присутні у коді відповідні конструкції. Наприклад:

- input(– використання введення;
- if / elif – розгалуження;

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				21
Змн.	Арк.	№ докум.	Підпис	Дата		

- % – перевірка подільності;
- .split() – розбиття рядка;
- max() – пошук максимуму;
- .reverse() або[::-1] – обернення списку.

На основі цього формується concept coverage:

$$ConceptCoverage = \frac{MatchedConcepts}{TotalConcepts}$$

Static analysis корисний для виявлення структурних проблем і відсутніх елементів реалізації, але він не гарантує правильну поведінку програми. Тому ця модель використовується переважно як допоміжна.

Четверта модель – hybrid_rule_based. Вона поєднує weighted tests і static analysis. Основою оцінки є weighted pass rate, а статичний аналіз використовується для додаткових штрафів.

Загальна логіка така:

$$BaseScore = WeightedCorrectness + Completeness + CodeQuality$$

$$FinalScore = BaseScore - StaticPenalty$$

StaticPenalty застосовується у випадках, коли: відсутні важливі концепти, concept coverage нижчий за порогове значення, виявлено потенційно небезпечні або помилкові конструкції, є помилка виконання. Перевага гібридного підходу полягає в тому, що він зберігає об'єктивність тестового підходу, але додатково враховує структуру рішення. У подальшому експериментальному аналізі цей підхід використовується як найсильніший детермінований rule-based підхід.

П'ята модель – llm_only_grading. У цьому підході велика мовна модель отримує лише умову задачі, код студента та рубрику оцінювання. Модель повинна повернути JSON із прогнозованою оцінкою, критеріями, поясненнями та risk flags.

Фрагмент промпту має такий зміст:

You are an AI assistant for grading introductory Python programming assignments.
Grade the student's code from 0 to 10.
Use the rubric:
- correctness: 0 to 5 points
- completeness: 0 to 3 points

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		22

- code_quality: 0 to 2 points
Return only raw JSON.

Перевага LLM-only моделі – здатність формувати зрозумілі пояснення. Недолік – відсутність фактичних доказів виконання коду. Через це модель може неправильно оцінити код, який виглядає логічним, але не проходить тести.

Шоста модель – test_aware_llm_grading. Вона є більш надійною LLM-версією, оскільки модель отримує не тільки код, а й результати виконання тестів. До промпту додаються: кількість пройдених тестів; загальна кількість тестів; прийнята вага; загальна вага; середній відсоток успішності; список провалених тест-кейсів; очікуваний і фактичні вихідні дані; повідомлення про помилку, якщо вона є.

Приклад частини промпту:

Test summary:
Passed tests: 3 / 4
Passed weighted score: 7.5 / 10
Weighted pass rate: 0.75

Failed test cases:
Input: 1001
Expected output: 950.95
Actual output: 970.97
Error:
Weight: 2.5

Цей baseline дозволяє LLM пояснювати оцінку на основі конкретних доказів. Наприклад, модель може вказати, що код неправильно обробляє суму покупки понад 1000 грн через неправильний порядок умов.

Сьома модель – test_aware_llm_guarded_grading. Вона використовує результат test-aware LLM, але після цього застосовує детерміновані обмежувачі. Вони обмежують максимальну оцінку у ризикових ситуаціях.

У роботі використано такі правила:

weightedPassRate == 0 – максимальний бал 2;

виникла помилка виконання – максимальний бал 4;

weightedPassRate < 0.5 – максимальний бал 8;

weightedPassRate < 0.75 – максимальний бал 8.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				23
Змн.	Арк.	№ докум.	Підпис	Дата		

Такі обмеження потрібні для зменшення небезпечного переоцінювання. Наприклад, якщо код має runtime error, але LLM випадково поставила високий бал, це обмежує фінальну оцінку.

Результат застосування обмежень додається як окремий критерій:

```
{
  "name": "guardrail_adjustment",
  "score": -2,
  "maxScore": 0,
  "explanation": "Original LLM score 6 was capped to 4. Applied rules: runtime_error_cap_4."
}
```

Ця модель не завжди покращує середню точність, але підвищує безпечність системи, оскільки зменшує ризик виставлення високої оцінки за код із критичними помилками.

Висновки до другого розділу

У другому розділі було описано практичну реалізацію сервісу AI-assisted оцінювання студентських програмних робіт. Було сформовано контрольний dataset на основі лабораторних завдань з Python, створено набір тест-кейсів із вагами та підготовлено варіанти програмних подань різної якості.

Розроблена архітектура сервісу базується на модульному pipeline, де кожен метод оцінювання реалізовано як окремий baseline через спільний інтерфейс IEvaluationBaseline. Це дозволяє легко додавати нові методи та порівнювати їх за єдиними метриками. У наступному розділі буде проведено експериментальний аналіз якості реалізованих методів.

РОЗДІЛ 3. АНАЛІЗ ЯКОСТІ МЕТОДІВ ОЦІНЮВАННЯ

3.1 Метрики якості та їх обґрунтування

Для оцінювання якості реалізованих методів було використано контрольний набір даних, що містить 16 програмних подань для 4 навчальних завдань з Python. Для кожного подання задано очікувану експертну оцінку в межах шкали від 0 до 10 балів. Завдання експерименту полягає у порівнянні прогнозованої оцінки, сформованої різними підходами, з очікуваною оцінкою.

Для LLM-підходів було використано моделі через OpenRouter API. Основною моделлю для порівняння стала nvidia/nemotron-3-super-120b-a12b:free. Додатково було протестовано моделі liquid/lfm-2.5-1.2b-instruct:free та google/gemma-4-31b-it:free для аналізу впливу вибору моделі на якість оцінювання. У всіх експериментах показник successRate дорівнював 1, тобто всі 16 подань були успішно оброблені без втрати результатів.

Основною метрикою якості є MAE – середня абсолютна помилка між очікуваною та прогнозованою оцінкою:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

де y_i — очікувана оцінка, $\hat{y}_i$ — прогнозована оцінка, n — кількість подань.

Також використовуються додаткові метрики:

$$ExactMatch = \frac{Count(\hat{y}_i = y_i)}{n}$$

$$WithinOne = \frac{Count(|\hat{y}_i - y_i| \leq 1)}{n}$$

Метрика Exact Match показує частку випадків, де прогнозована оцінка повністю збіглася з очікуваною. Within-One метрика показує частку випадків, де похибка не перевищила одного бала. Така метрика є важливою для навчального контексту, оскільки різниця в межах одного бала може вважатися прийнятною для попереднього III оцінювання, якщо фінальне рішення все одно залишається за викладачем.

Окрім цього, у роботі використовуються метрики ризику:

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				25
Змн.	Арк.	№ докум.	Підпис	Дата		

Max Absolute Error – найбільша абсолютна помилка;

Large Error Rate – частка випадків, де похибка перевищує 2 бали;

Overestimation Rate – частка випадків, де система завищила оцінку;

Underestimation Rate – частка випадків, де система занижила оцінку;

Guardrail Applied Rate – частка випадків, де було застосовано детерміністичні обмеження.

Ці метрики важливі, оскільки середня помилка не завжди достатньо показує ризиковість методу. Наприклад, два методи можуть мати схожий MAE, але один частіше завищує слабкі роботи, а інший – навпаки, занижує частково правильні. Для освітнього оцінювання завищення слабких робіт є особливо небезпечним, тому рівень переоцінки розглядається як окрема характеристика якості.

3.2 Результати та порівняльний аналіз моделей

Зведені результати експериментів без використання Llm наведено в табл.

3.1.

Таблиця 3.1

Порівняння якості методів оцінювання без використання Llm

Метод	MAE	Exact Match	Within ± 1	Max Error	Large Error Rate	Overestimation	Underestimation
Equal-weight rule-based	1.33	0.25	0.69	4.75	0.19	0.50	0.25
Weighted rule-based	1.16	0.38	0.69	4.75	0.19	0.38	0.25
Static analysis rule-based	3.62	0.25	0.25	7.00	0.69	0.75	0.00
Hybrid rule-based	1.09	0.31	0.75	3.75	0.19	0.38	0.31

Результати показують, що найгіршим окремим підходом став static_analysis_rule_based: MAE дорівнює 3.62, large error rate = 0.69, а overestimation = 0.75. Це означає, що статичний аналіз як самостійний метод часто завищує оцінку, оскільки перевіряє наявність потрібних конструкцій у

коді, але не гарантує правильність поведінки програми. Наприклад, код може містити if, input, % або max, але при цьому реалізовувати неправильну логіку.

Порівняння equal_weight_rule_based і weighted_rule_based показує, що використання ваг тест-кейсів покращило середню абсолютну помилку з 1.33 до 1.16 і підвищило exact match метрику з 0.25 до 0.38. При цьому within-one метрика залишилася однаковою – 0.69. Це свідчить, що зважені тести краще узгоджуються з очікуваною оцінкою, особливо у випадках, де окремі граничні випадки мають більшу важливість для перевірки алгоритму.

Hybrid rule-based показав найкращий результат серед повністю детермінованих підходів: MAE = 1.09, within-one accuracy = 0.75, max error = 3.75. Це підтверджує доцільність поєднання поведінкової перевірки через тести зі статичними штрафами. Водночас статичний аналіз у цьому випадку використовується не як основа оцінювання, а лише як додатковий сигнал ризику.

Наступним розглянемо порівняння різних підходів із використанням Llm моделей, табл. 3.2.

Таблиця 3.2

Порівняння LLM-моделей у guarded test-aware режимі

Модель	MAE	Exact Match	Within ±1	Max Error	Overestimation	Underestimation	Guardrail Rate
nvidia/nemotron-3-super-120b-a12b:free	1.00	0.44	0.69	3.00	0.25	0.31	0.69
google/gemma-4-31b-it:free	1.06	0.38	0.75	3.00	0.19	0.44	0.69
liquid/lfm-2.5-1.2b-instruct:free	1.16	0.31	0.69	3.00	0.50	0.19	0.69

Модель nvidia/nemotron-3-super-120b-a12b:free показала найкращий MAE = 1.00, а також найвищий exact match показник серед трьох моделей – 0.44. Це означає, що вона найчастіше формувала оцінки, близькі до очікуваних, і краще узгоджувалась із заданою рубрикою.

Модель google/gemma-4-31b-it:free показала трохи гірший MAE = 1.06, але найкращий показник within-one = 0.75. Також вона має найнижчий overestimation rate = 0.19. Це означає, що Gemma у guarded-режимі була більш консервативною.

Вона рідше завищувала оцінки, але частіше їх занижувала, що видно з $\text{underestimation rate} = 0.44$.

Модель `liquid/lfm-2.5-1.2b-instruct:free` показала найслабший результат серед трьох моделей: $\text{MAE} = 1.16$, $\text{exact match accuracy} = 0.31$, $\text{overestimation rate} = 0.50$. Це свідчить, що швидша або менша модель може бути менш стабільною в задачі оцінювання коду, особливо коли потрібно правильно інтерпретувати `failed test cases` і не завищувати бал.

Отримані результати підтверджують, що вибір LLM-моделі має суттєвий вплив на якість оцінювання. При цьому більша модель не завжди є однозначно кращою за всіма метриками: Nemotron має найкращий MAE, Gemma – найкращий within-one accuracy і найнижче завищення, а Liquid демонструє вищу схильність до переоцінювання.

3.3 Аналіз помилок та інтерпретація

Для кращого розуміння результатів було розглянуто окремі випадки, де методи давали суттєво різні оцінки.

Одним із найпоказовіших прикладів є подання `S_L2_T3_004`. У цьому випадку код проходить частину тестів, але має помилку виконання у гілці для суми понад 1000 грн через використання невизначеної змінної `total`. Rule-based підходи, які спираються переважно на частку пройдених тестів, переоцінювали це рішення. Наприклад, `weighted rule-based` дав прогнозовану оцінку 5.75 при очікуваній оцінці 1.

Test-aware LLM також переоцінила цей випадок без `guardrails`: прогнозована оцінка становила 7.5, а абсолютна помилка – 6.5. Це показує, що навіть за наявності тестових результатів LLM може надати надто високу оцінку, якщо частина тестів проходить успішно.

Guarded LLM у цьому випадку застосувала правило `runtime_error_cap_4`, яке обмежило максимальну оцінку до 4 балів. Це не зробило оцінку ідеальною, але суттєво зменшило ризик критичного завищення. Саме цей приклад демонструє практичну користь детерміністичних обмежень: вони не замінюють LLM, але обмежують її потенційно небезпечні рішення.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				28
Змн.	Арк.	№ докум.	Підпис	Дата		

Загалом аналіз помилок показує що тестові підходи добре оцінюють функціональну поведінку, але можуть переоцінювати код, який випадково проходить частину тестів. Водночас можна зробити висновок, що hybrid rule-based підхід є сильним детерміністичним підходом. Проте, загалом LLM-підходи дають кращі пояснення, але можуть помилятися в оцінці та використання обмежень при цьому зменшують максимальні помилки та ризик небезпечного завищення;

3.4 Обмеження роботи та напрями розвитку

Попри позитивні результати, проведений експеримент має низку обмежень. По-перше, контрольний датасет є невеликим і містить лише 16 подань. Цього достатньо для перевірки архітектури сервісу та первинного порівняння підходів, але недостатньо для статистично сильних висновків про якість системи на реальних студентських роботах.

Варто також зауважити, що очікуваний бал було сформовано вручну. Це дозволяє контролювати експеримент, але створює ризик суб'єктивності. У майбутньому доцільно використати оцінки кількох викладачів або реальні анонімізовані студентські роботи.

Також, LLM-моделі можуть бути нестабільними. Вони іноді повертають некоректний JSON, потребують retry-механізмів, можуть потрапляти під rate limit або давати різні відповіді залежно від моделі. У роботі для цього було додано retry policy та обробку форматних помилок, але для production-рівня потрібне додаткове логування, моніторинг і аудит.

Окремим напрямом розвитку є відповідність AI-системи вимогам human-in-the-loop. Усі реалізовані підходи повертають політику teacher_must_confirm, що означає – система формує рекомендацію, але не приймає фінальне рішення. У майбутньому цей підхід можна розширити механізмами аудиту, пояснюваності, логуванням усіх рішень, можливістю ручної заміни та аналізом правдивості.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				29
Змн.	Арк.	№ докум.	Підпис	Дата		

Висновки до третього розділу

У третьому розділі було проведено експериментальний аналіз якості реалізованих методів AI-assisted оцінювання студентських програмних робіт. Було використано набір із 16 контрольних подань і порівняно rule-based, static-analysis, hybrid та LLM-based підходи.

Загалом результати підтверджують, що найефективнішим є не використання одного окремого методу, а гібридний підхід, який поєднує тестування, пояснюваний аналіз коду, LLM reasoning та обмеження безпеки. Такий підхід відповідає концепції ІІІ оцінювання, де система підтримує викладача, але не замінює його фінальне рішення.

		Осипчук А.О.			ІІІЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				30
Змн.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ

У ході виконання курсової роботи було проведено комплексне дослідження та практичну реалізацію сервісу пояснюваного III оцінювання студентських програмних робіт. Основною метою роботи було порівняти різні підходи до формування попередньої оцінки за програмний код.

У першому розділі було проаналізовано задачу автоматизованого оцінювання програмних робіт та визначено її основні обмеження. Тут було описано звичайні автоматичні тести є швидким і відтворюваним способом перевірки функціональної коректності, та окрему увагу приділено LLM-based оцінюванню, яке дає змогу формувати пояснення до оцінки.

У другому розділі було спроектовано та реалізовано прототип сервісу III оцінювання на базі .NET / ASP.NET Core Web API. Було сформовано контрольний набір даних на основі навчальних лабораторних завдань з Python, що включає завдання, тест-кейси з вагами та програмні подання різної якості. Реалізовано порядок виконання студентського коду, запуску тестів, формування результатів оцінювання та обчислення метрик якості.

У третьому розділі було проведено експериментальний аналіз реалізованих методів. Результати показали, що статичний аналіз як самостійний метод є найменш надійним для виставлення оцінки, оскільки має високу середню помилку та схильність до переоцінювання. Зважені тест-кейси покращили результат порівняно з рівнозначними тестами, а гібридний rule-based підхід став найкращим серед повністю детермінованих методів.

Проведений аналіз підтвердив, що найефективнішим є не використання одного окремого методу, а комбінований підхід. Автоматичні тести забезпечують об'єктивну перевірку поведінки програми, статичний аналіз допомагає виявити структурні проблеми, LLM формує пояснення та аналізує логіку коду, а обмеження зменшують ризик небезпечного переоцінювання. Саме таке поєднання є найбільш доцільним для освітнього сценарію, де важлива не лише точність оцінки, а й її пояснюваність, прозорість і можливість перегляду викладачем.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				31
Змн.	Арк.	№ докум.	Підпис	Дата		

Таким чином, поставлені завдання курсової роботи було виконано повністю. Було проаналізовано предметну область, спроектовано архітектуру сервісу, реалізовано набір підходів оцінювання, інтегровано LLM через OpenRouter API та проведено експериментальне порівняння методів. Розроблений прототип може бути використаний як основа для подальшого розвитку системи оцінювання студентських програмних робіт, зокрема для інтеграції в більшу освітню платформу або майбутню магістерську роботу.

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				32
Змн.	Арк.	№ докум.	Підпис	Дата		

СПИСОК ЛІТЕРАТУРИ

1. EU AI Act : Official Journal of the European Union від, № 1689 : станом на 12.07.2024.
2. Create web APIs with ASP.NET Core. *learn.microsoft.com*. URL: <https://learn.microsoft.com/en-us/aspnet/core/web-api/?view=aspnetcore-10.0>
3. mean_absolute_error. *scikit-learn.org*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html
4. Subprocess — Subprocess management. *Docs.python.org*. URL: <https://docs.python.org/3/library/subprocess.html>
5. Get started with OpenRouter. *Openrouter.ai*. URL: <https://openrouter.ai/docs/quickstart>
6. Alkafaween U., Albluwi I., Denny P. Automating Autograding: Large Language Models as Test Suite Generators for Introductory Programming. *Journal of Computer Assisted Learning*. Т. 41.
7. Human-in-the-Loop LLM Grading for Handwritten Mathematics Assessments / Arne Vanhoyweghen та ін. *Education and Learning Technologies*.
8. Jukiewicz M. A. A systematic comparison of Large Language Models for automated assignment assessment in programming education: Exploring the importance of architecture and vendor. *Computers and Education Open*. 2025. Т. 10.
9. CodEv: An Automated Grading Framework Leveraging Large Language Models for Consistent and Constructive Feedback / E.-Q. Tseng та ін.
10. Unveiling Scoring Processes: Dissecting the Differences Between LLMs and Human Graders in Automatic Scoring / Xuansheng Wu та ін. *Technology, Knowledge and Learning*.
11. LLM-Based Automated Grading with Human-in-the-Loop / Yucheng Chu та ін. *2025 IEEE International Conference*.

		Осипчук А.О.			ІПЗМ.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		33

ДОДАТКИ

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				34
Змн.	Арк.	№ докум.	Підпис	Дата		

BatchEvaluationSummary.cs

```

public class BatchEvaluationSummary
{
    public string BaselineName { get; set; } = string.Empty;

    public int TotalSubmissions { get; set; }
    public int FailedEvaluations { get; set; }
    public double SuccessRate { get; set; }

    public double AverageAbsoluteError { get; set; }
    public double ExactMatchAccuracy { get; set; }
    public double WithinOneAccuracy { get; set; }

    public double MaxAbsoluteError { get; set; }
    public double LargeErrorRate { get; set; }

    public double OverestimationRate { get; set; }
    public double UnderestimationRate { get; set; }

    public int OverestimatedCount { get; set; }
    public int UnderestimatedCount { get; set; }
    public int ExactMatchCount { get; set; }
    public int WithinOneCount { get; set; }
    public int LargeErrorCount { get; set; }

    public double AveragePredictedScore { get; set; }
    public double AverageExpectedScore { get; set; }

    public int GuardrailAppliedCount { get; set; }
    public double GuardrailAppliedRate { get; set; }

    public List<EvaluationGrade> Grades { get; set; } = [];
}

```

CriterionScore.cs

```

public class CriterionScore
{
    public string Name { get; set; } = string.Empty;
    public double Score { get; set; }
    public double MaxScore { get; set; }
    public string Explanation { get; set; } = string.Empty;
}

```

EvaluationGrade.cs

```

public class EvaluationGrade
{
    public string BaselineName { get; set; } = string.Empty;
}

```

		Осипчук А.О.			ІПЗм.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		35

```

public string ModelName { get; set; } = string.Empty;

public string SubmissionId { get; set; } = string.Empty;
public string AssignmentId { get; set; } = string.Empty;

public int MaxScore { get; set; }
public int ExpectedScore { get; set; }

public double PredictedScore { get; set; }
public double AbsoluteError { get; set; }

public int PassedTests { get; set; }
public int TotalTests { get; set; }

public double PassedWeight { get; set; }
public double TotalWeight { get; set; }
public double PassRate { get; set; }

public List<CriterionScore> Criteria { get; set; } = [];
public List<string> RiskFlags { get; set; } = [];

public string FinalDecisionPolicy { get; set; } = "teacher_must_confirm";
}

```

DatasetService.cs

```

public class DatasetService
{
    private readonly List<Assignment> _assignments = [];
    private readonly List<Submission> _submissions = [];
    private readonly List<TestCase> _testCases = [];

    public IReadOnlyList<Assignment> Assignments => _assignments;
    public IReadOnlyList<Submission> Submissions => _submissions;
    public IReadOnlyList<TestCase> TestCases => _testCases;

    public void Load()
    {
        _assignments.Clear();
        _submissions.Clear();
        _testCases.Clear();

        LoadAssignments();
        LoadSubmissions();
        LoadTestCases();
    }

    private void LoadAssignments()
    {
        var path = Path.Combine(AppContext.BaseDirectory, "Dataset",
            "assignments_with_required_concepts.csv");
    }
}

```

		Осипчук А.О.			ІІЗМ.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		36

```

using var reader = new StreamReader(path);
using var csv = new CsvReader(reader, CreateCsvConfig());

var records = csv.GetRecords<Assignment>().ToList();
_assignmentments.AddRange(records);
}

private void LoadTestCases()
{
    var path = Path.Combine(AppContext.BaseDirectory, "Dataset", "test_cases_weighted.csv");

    using var reader = new StreamReader(path);
    using var csv = new CsvReader(reader, CreateCsvConfig());

    var records = csv.GetRecords<TestCase>().ToList();
    _testCases.AddRange(records);
}

private void LoadSubmissions()
{
    var path = Path.Combine(AppContext.BaseDirectory, "Dataset", "submissions.jsonl");

    foreach (var line in File.ReadLines(path))
    {
        if (string.IsNullOrEmpty(line))
            continue;

        var submission = JsonSerializer.Deserialize<Submission>(
            line,
            new JsonSerializerOptions
            {
                PropertyNameCaseInsensitive = true
            });

        if (submission is not null)
            _submissions.Add(submission);
    }
}

public Assignment? GetAssignment(string assignmentId)
{
    return _assignments.FirstOrDefault(x => x.AssignmentId == assignmentId);
}

public Submission? GetSubmission(string submissionId)
{
    return _submissions.FirstOrDefault(x => x.SubmissionId == submissionId);
}

public List<TestCase> GetTestCases(string assignmentId)
{
    return _testCases

```

```

        .Where(x => x.AssignmentId == assignmentId)
        .ToList();
    }

    private static CsvConfiguration CreateCsvConfig()
    {
        return new CsvConfiguration(CultureInfo.InvariantCulture)
        {
            HeaderValidated = null,
            MissingFieldFound = null,
            PrepareHeaderForMatch = args =>
                args.Header.Replace("_", "").ToLowerInvariant()
        };
    }
}

```

PythonExecutionService.cs

```

public class PythonExecutionService
{
    private const int TimeoutMs = 3000;

    public async Task<TestRunResult> RunSubmissionTestsAsync(
        Submission submission,
        List<TestCase> testCases,
        CancellationToken cancellationToken = default)
    {
        var result = new TestRunResult
        {
            SubmissionId = submission.SubmissionId,
            AssignmentId = submission.AssignmentId,
            Total = testCases.Count,
            TotalWeight = testCases.Sum(x => x.Weight <= 0 ? 1 : x.Weight)
        };

        foreach (var testCase in testCases)
        {
            var caseResult = await RunSingleTestAsync(submission.StudentCode, testCase,
                cancellationToken);

            result.Results.Add(caseResult);

            if (caseResult.Passed)
            {
                result.Passed++;
                result.PassedWeight += caseResult.Weight;
            }
        }

        return result;
    }
}

```

		Осипчук А.О.			ІІЗМ.КР.Б-125-26-ІІЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		38

```

private static async Task<TestCaseResult> RunSingleTestAsync(
    string code,
    TestCase testCase,
    CancellationToken cancellationToken)
{
    var tempFile = Path.Combine(Path.GetTempPath(), $"submission_{Guid.NewGuid():N}.py");

    await File.WriteAllTextAsync(tempFile, code, cancellationToken);

    try
    {
        var startInfo = new ProcessStartInfo
        {
            FileName = "python",
            Arguments = $"\"{tempFile}\"",
            RedirectStandardInput = true,
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false,
            CreateNoWindow = true
        };

        using var process = new Process
        {
            StartInfo = startInfo
        };

        process.Start();

        await process.StandardInput.WriteAsync(testCase.InputData);
        process.StandardInput.Close();

        var outputTask = process.StandardOutput.ReadToEndAsync(cancellationToken);
        var errorTask = process.StandardError.ReadToEndAsync(cancellationToken);

        var exited = await WaitForExitAsync(process, TimeoutMs, cancellationToken);

        if (!exited)
        {
            TryKill(process);
        }

        return new TestCaseResult
        {
            Input = testCase.InputData,
            ExpectedOutput = Normalize(testCase.ExpectedOutput),
            ActualOutput = "",
            Error = "Execution timeout.",
            Passed = false,
            Weight = testCase.Weight <= 0 ? 1 : testCase.Weight,
        };
    }
}

```

		Осипчук А.О.			ІІЗМ.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		39

```

var output = await outputTask;
var error = await errorTask;

var actual = Normalize(output);
var expected = Normalize(testCase.ExpectedOutput);

return new TestCaseResult
{
    Input = testCase.InputData,
    ExpectedOutput = expected,
    ActualOutput = actual,
    Error = error.Trim(),
    Passed = string.IsNullOrEmpty(error) && IsOutputCorrect(actual, expected),
    Weight = testCase.Weight <= 0 ? 1 : testCase.Weight,
};
}
finally
{
    if (File.Exists(tempFile))
        File.Delete(tempFile);
}
}

private static bool IsOutputCorrect(string actual, string expected)
{
    actual = Normalize(actual).ToLowerInvariant();
    expected = Normalize(expected).ToLowerInvariant();

    var expectedParts = expected
        .Split('|', StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries);

    if (expectedParts.Length == 0)
        return false;

    return expectedParts.All(part => actual.Contains(part));
}

private static async Task<bool> WaitForExitAsync(
    Process process,
    int timeoutMs,
    CancellationToken cancellationToken)
{
    var waitTask = process.WaitForExitAsync(cancellationToken);
    var delayTask = Task.Delay(timeoutMs, cancellationToken);

    var completedTask = await Task.WhenAny(waitTask, delayTask);

    return completedTask == waitTask;
}

private static void TryKill(Process process)
{

```

```

    try
    {
        if (!process.HasExited)
            process.Kill(entireProcessTree: true);
    }
    catch
    {
        // ignore
    }
}

private static string Normalize(string value)
{
    return value
        .Replace("\r", "")
        .Trim();
}
}

```

RuleBasedGradingService.cs

```

public class RuleBasedGradingService
{
    public RuleBasedGrade Grade(
        Assignment assignment,
        Submission submission,
        TestRunResult testRun)
    {
        var passRate = testRun.Total == 0
            ? 0
            : (double)testRun.Passed / testRun.Total;

        var correctnessScore = Math.Round(passRate * 5, 2);

        var hasRuntimeError = testRun.HasRuntimeError;
        var qualityScore = hasRuntimeError ? 0 : EstimateCodeQuality(submission.StudentCode);
        var completenessScore = EstimateCompleteness(submission.StudentCode, passRate);

        var predictedScore = correctnessScore + completenessScore + qualityScore;
        predictedScore = Math.Round(Math.Min(predictedScore, assignment.MaxScore), 2);

        var riskFlags = new List<string>();

        if (hasRuntimeError)
            riskFlags.Add("runtime_error");

        if (passRate < 1)
            riskFlags.Add("failed_tests");

        if (submission.StudentCode.Length < 40)
            riskFlags.Add("too_short_solution");
    }
}

```

```

riskFlags.Add("teacher_must_review");

return new RuleBasedGrade
{
    SubmissionId = submission.SubmissionId,
    AssignmentId = submission.AssignmentId,
    MaxScore = assignment.MaxScore,
    ExpectedScore = submission.ExpectedScore,
    PredictedScore = predictedScore,
    AbsoluteError = Math.Abs(predictedScore - submission.ExpectedScore),
    PassedTests = testRun.Passed,
    TotalTests = testRun.Total,
    RiskFlags = riskFlags,
    Criteria =
    [
        new CriterionScore
        {
            Name = "correctness",
            Score = correctnessScore,
            MaxScore = 5,
            Explanation = $"Passed {testRun.Passed} out of {testRun.Total} tests."
        },
        new CriterionScore
        {
            Name = "completeness",
            Score = completenessScore,
            MaxScore = 3,
            Explanation = "Estimated from test coverage and solution structure."
        },
        new CriterionScore
        {
            Name = "code_quality",
            Score = qualityScore,
            MaxScore = 2,
            Explanation = "Estimated from basic code quality heuristics."
        }
    ]
};
}

private static double EstimateCompleteness(string code, double passRate)
{
    if (passRate >= 1.0)
        return 3;

    if (passRate >= 0.75)
        return 2;

    if (passRate >= 0.5)
        return 1.5;

    if (passRate > 0)

```

		Осипчук А.О.			ІІЗМ.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		42

```

        return 1;

    return 0;
}

private static double EstimateCodeQuality(string code)
{
    var score = 2.0;

    if (code.Contains("while True"))
        score -= 0.5;

    if (code.Length < 40)
        score -= 1;

    if (!code.Contains("input"))
        score -= 0.5;

    return Math.Max(0, score);
}
}

```

EvaluationGrade.cs

```

public class EvaluationGrade
{

```

EvaluationGrade.cs

```

public class EvaluationGrade
{

```

EvaluationGrade.cs

```

public class EvaluationGrade
{

```

		Осипчук А.О.			ІПЗМ.КР.Б-125-26-ПЗ	Арк.
		Марчук Г.В.				43
Змн.	Арк.	№ докум.	Підпис	Дата		